

Fundamentals of RISC-V ISA	1
1. Title	1
2. Objective	2
3. Theory (Multi-Level)	2
3.1 Beginner — Simple Intuition	2
3.2 Intermediate — Engineering View	2
3.3 Advanced — Signal/Architecture Level	2
4. Architecture / Concept	3
PicoRV32 CPU Block Diagram	3
Instruction Flow (Fetch → Decode → Execute → Memory → Writeback)	4
5. Implementation	4
Files in This Module	4
How It Works in This Project	4
6. Lab / Hands-on Section	4
Prerequisites	4
Step-by-Step Commands	4
7. Waveform / Verification Insight	5
What to Observe in GTKWave	5
Key Observations	5
ROM Content at Boot	6
8. Common Issues & Debug Tips	6
Debug Tip: Check Instruction Encoding	6
9. Summary	6

Fundamentals of RISC-V ISA

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

Fundamentals of RISC-V ISA — Understanding Instruction Execution with PicoRV32

2. Objective

By the end of this module, learners will be able to:

- Explain the RISC-V base integer ISA (RV32I) and its register file
- Describe how instructions are encoded in 32-bit words
- Observe PicoRV32 CPU executing instructions through simulation
- Understand the native memory interface (mem_valid/mem_ready) used by PicoRV32
- Read and interpret basic RISC-V waveforms in GTKWave

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

Think of RISC-V as a set of very simple cooking instructions your CPU follows, one at a time:

- `ADD x1, x2, x3` → “Take the number in shelf x2, add the number in shelf x3, put the result on shelf x1”
- `LW x4, 0(x1)` → “Go to the address in x1, pick up a 32-bit value, put it in x4”

The CPU (PicoRV32) has **32 shelves** (registers, named `x0` – `x31`) and runs through instructions stored in memory one by one. It is a **RISC** (Reduced Instruction Set Computer) processor — meaning each instruction is small, simple, and takes a predictable amount of time.

3.2 Intermediate — Engineering View

RV32I Base ISA defines: - **32 general-purpose registers** (`x0`–`x31`), each 32 bits wide. `x0` is hardwired to zero. - **6 core instruction formats**: R, I, S, B, U, J — all 32 bits wide -

Instructions: load/store, arithmetic/logical, branches, jumps, and system calls

Instruction Format	Fields	Example Instructions
R-type	opcode, rd, funct3, rs1, rs2, funct7	ADD, SUB, AND
I-type	opcode, rd, funct3, rs1, imm[11:0]	ADDI, LW, JALR
S-type	opcode, imm[4:0], funct3, rs1, rs2, imm[11:5]	SW, SB
B-type	opcode, offset, funct3, rs1, rs2	BEQ, BNE
U-type	opcode, rd, imm[31:12]	LUI, AUIPC
J-type	opcode, rd, imm[20:1]	JAL

Program Counter (PC): Starts at 0x00000000 (PROGADDR_RESET in PicoRV32). Every instruction fetch increments PC by 4 bytes.

3.3 Advanced — Signal/Architecture Level

PicoRV32 uses a **native memory interface** (not AXI at this stage):

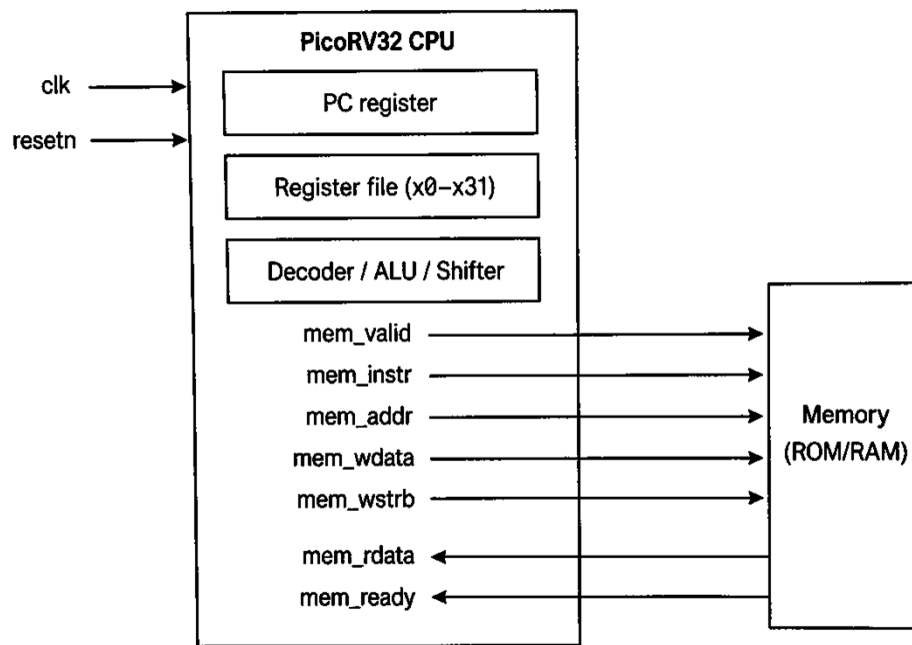
Signal	Direction	Width	Description
-----	-----	-----	-----
mem_valid	CPU→Mem	1	CPU asserts when address is valid
mem_instr	CPU→Mem	1	High for instruction fetch
mem_ready	Mem→CPU	1	Memory asserts when data is ready
mem_addr	CPU→Mem	32	Address being accessed
mem_wdata	CPU→Mem	32	Write data
mem_wstrb	CPU→Mem	4	Byte write strobe (0xF=full word)
mem_rdata	Mem→CPU	32	Read data returned to CPU

The handshake is simple: CPU asserts mem_valid, keeps it high; memory processes the request and asserts mem_ready for one cycle; CPU latches mem_rdata. This is a **stall-based** protocol — if memory is slow, mem_ready stays low and the CPU pipeline stalls.

In the standalone PicoRV32 test (rtl/PicoRV32/), a simple on-chip memory answers immediately, making mem_ready follow mem_valid in one cycle.

4. Architecture / Concept

PicoRV32 CPU Block Diagram



Key design parameters (from rtl/top.v instantiation):

```
.ENABLE_COUNTERS    (1),    // Cycle/instret counters enabled
.ENABLE_REGS_16_31  (1),    // Full 32-register file
.CATCH_MISALIGN     (1),    // Trap misaligned accesses
.CATCH_ILLINSN      (1),    // Trap illegal instructions
.PROGADDR_RESET     (32'h0000_0000), // Boot address
.STACKADDR           (32'h0001_00FC) // Stack top
```

Instruction Flow (Fetch → Decode → Execute → Memory → Writeback)

Cycle 1: mem_valid=1, mem_instr=1, mem_addr=0x0000_0000 (Instruction Fetch)
← mem_ready=1, mem_rdata=0x00020137 (LUI x2, 0x20)

Cycle 2: Decode 0x00020137 → LUI x2, imm=0x20000
Execute: x2 ← 0x0002_0000

Cycle 3: mem_valid=1, mem_instr=1, mem_addr=0x0000_0004 (Next IF)
← mem_ready=1, mem_rdata=0xFFC10113 (ADDI x2, x2, -4)

5. Implementation

Files in This Module

File	Role
rtl/PicoRV32/picorv32.v	Main PicoRV32 CPU RTL (the processor itself)
rtl/PicoRV32/Memory.v	Simple instruction + data memory
rtl/PicoRV32/top.v	Top wrapper connecting CPU to memory
rtl/PicoRV32/tb_processor.v	Testbench for isolated PicoRV32 simulation

How It Works in This Project

- The standalone test (make pico) runs PicoRV32 with a dedicated instruction memory containing a simple RISC-V assembly program (arithmetic, loads, stores).
- The testbench (tb_processor.v) monitors mem_valid/mem_ready and dumps signals to tb_picorv32.vcd.
- The firmware used in Sub-modules 7/8 is compiled by riscv32-unknown-elf-gcc targeting RV32I and loaded into rom.hex.

6. Lab / Hands-on Section

Prerequisites

```
# Ensure tools are installed
verilator --version # >= 4.220
gtkwave --version
riscv32-unknown-elf-gcc -version
```

Step-by-Step Commands

Step 1: Navigate to project root

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
```

Step 2: Run the PicoRV32 standalone simulation

```
make pico
```

This does: 1. Builds the standalone RISC-V firmware in rtl/PicoRV32/ 2. Compiles all RTL + testbench with Verilator (--binary --trace --timing) 3. Runs the simulation binary → produces tb_picorv32.vcd 4. Opens GTKWave automatically

Step 3: Observe the simulation output

Expected terminal output:

```
[ROM] loaded from parameter: ...  
-- Simulation Complete --
```

Step 4: View waveforms

```
make wave
```

or directly:

```
gtkwave tb_picorv32.vcd &
```

Step 5: Clean build artifacts

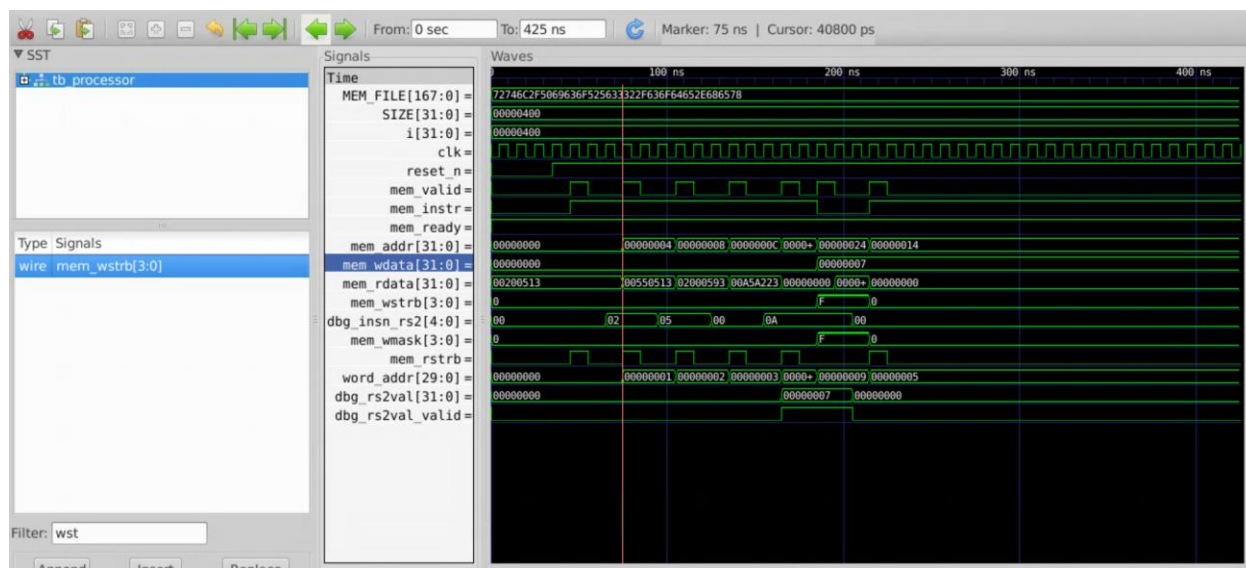
```
make clean
```

7. Waveform / Verification Insight

What to Observe in GTKWave

Open tb_picorv32.vcd and add these signals to the waveform window:

```
clk
resetsn
mem_valid
mem_instr
mem_ready
mem_addr [31:0]
mem_wdata [31:0]
mem_wstrb [3:0]
mem_rdata [31:0]
```



Key Observations

Signal Pattern	What It Means
mem_valid=1, mem_instr=1	CPU is fetching an instruction
mem_valid=1, mem_instr=0, mem_wstrb≠0	CPU is performing a store
mem_valid=1, mem_instr=0, mem_wstrb=0	CPU is performing a load
mem_ready=0 for multiple cycles	Memory stall (pipeline waits)
mem_addr incrementing by 4	Sequential instruction execution

ROM Content at Boot

The first instruction at 0x00000000 is 0x00020137 = LUI x2, 0x20, which sets up the stack pointer to 0x00020000. You can verify this in the waveform by checking mem_rdata when mem_addr=0x00000000.

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
make pico fails — tool not found	Verilator/GCC not in PATH	Source your tools: source ~/oss/environment
tb_picorv32.vcd not generated	Verilator --trace missing	Check VLFLAGS in Makefile includes --trace
Simulation ends immediately	ROM is all NOPs (0x00000013)	Check the firmware was compiled in rtl/PicoRV32/
GTKWave shows flat signals	Wrong VCD file opened	Confirm file path; use ls -la *.vcd
CPU stalls forever	mem_ready never asserts	Check memory module wiring in top.v
trap signal goes high	Illegal instruction or misaligned access	Check firmware ELF for undefined behavior

Debug Tip: Check Instruction Encoding

To decode any mem_rdata value manually:

```
# Quick decoder (Python)
instr = 0x00020137
opcode = instr & 0x7F      # = 0x37 = LUI
rd      = (instr >> 7) & 0x1F # = x2
imm     = instr >> 12       # = 0x20
print(f"LUI x{rd}, 0x{imm:05X}")
```

9. Summary

Topic	Key Takeaway
RISC-V ISA	32-bit fixed-length instructions, 32 registers, 6 formats
PicoRV32	Open-source RV32I implementation, simple stall-based memory interface
Native memory bus	mem_valid/mem_ready handshake; stall if memory is slow
Boot address	PROGADDR_RESET = 0x00000000; stack at 0x0001_00FC
Lab result	tb_picorv32.vcd shows full instruction pipeline including IF, EX, MEM

This module establishes the foundation that all subsequent modules build upon. The PicoRV32 CPU's native memory interface (mem_valid/mem_ready) is converted to AXI-Lite in Sub-module 5 — this translation is the core bridge of the entire SoC Subsystem.

Next Sub-module → Sub-module 2: Introduction to AXI-Lite

echiphub.in